

A TITLE

JO MACMAHON

ABSTRACT. An abstract

1. TYPE THEORY

The basic objects of study in type theory are types, which are denoted by uppercase letters: A , B , etc. Unlike in set theory, every object in type theory has a type. In set-theoretic mathematics, we might make statements such as “*Let n be a natural number. Then n has a successor.*” What is meant by this is something like “let n denote something”, and then “if $n \in \mathbb{N}$, then n has a successor”. In type theory this is not allowed: every object we consider must be constrained by its type. Supposing we have defined a type $\mathbb{N}$ of natural numbers (which we will do later), we would write $n : \mathbb{N}$ to indicate that n is a natural number. We may then proceed with the claim about its successor.

In this way, we can view types as “containing” elements. Notationally, we write

$$x : A$$

to mean that x is an element of type A .

We will show how to construct types out of other types, but first we introduce the important notion of type universes.

1.1. Universes

In order to avoid a situation similar to Russell’s paradox, we define a hierarchy of **universes**, denoted

$$\mathcal{U}_0 : \mathcal{U}_1 : \dots$$

Each $\mathcal{U}_i$ is an element of $\mathcal{U}_{i+1}$, and furthermore every universe contains all the types contained in previous universes. I.e. if $x : \mathcal{U}_i$ then $x : \mathcal{U}_j$ for all $j \geq i$. This is known as the **cumulative** property. When we say A is a type, what we mean is that $A : \mathcal{U}_i$ for some universe $\mathcal{U}_i$. When working with (a finite number of) types in different universes, the cumulative property guarantees that we can always find a universe in which all our types are present.

1.2. Function types

with the following rules: supposing A and B are types,

- There is a **function type** $A \rightarrow B$.
- There is a **product type** $A \times B$.
- There is a **coproduct type** $A + B$.

—

- Judgemental equality vs propositional equality: $\equiv$ vs $=$
- Function types
 - Definition in closed form or open form

1.3. Type families

Functions of type $B : A \rightarrow \mathcal{U}$, i.e. for $x : A$, we have $B(x)$ a type.

1.4. Dependent functions

Functions whose output type depends on the input (type or value). Given a type A and a family $B : A \rightarrow \mathcal{U}$, write

$$f : \prod_{x:A} B(x)$$

to mean a function which takes a parameter x of type A and returns a value of type $B(x)$.

1.5. Product types

1.5.1. “*Recurzor*”

- Generic way to construct functions taking product parameters, using simple functions

$$\begin{aligned} \text{rec}_{A \times B} &: \prod_{C:\mathcal{U}} (A \rightarrow B \rightarrow C) \rightarrow A \times B \rightarrow C \\ \text{rec}_{A \times B}(C, g, (a, b)) &:\equiv g(a)(b) \end{aligned}$$

Projections:

$$\begin{aligned} \pi_1 &:\equiv \text{rec}_{A \times B}(A, \lambda a . \lambda b . a) \\ \pi_2 &:\equiv \text{rec}_{A \times B}(B, \lambda a . \lambda b . b) \end{aligned}$$

REFERENCES

DEPARTMENT OF MATHEMATICS, UNIVERSITY OF MANCHESTER, OXFORD RD, MANCHESTER
M13 9PL, UNITED KINGDOM
Email address: jo.macmahon@postgrad.manchester.ac.uk